

SciNumTools DIPH5

Environment HDF5 Format Specification

Version 1.0

Contents

1	Scope	1
2	Persisted environment boundary	1
3	File identification and versioning	2
3.1	File naming	2
4	Paths and objects	2
4.1	Collections	2
5	Value datasets	3
5.1	Value type table	3
6	Node settings and provenance	3
7	Examples	3
7.1	Scalar values and hierarchy	3
7.2	Arrays and units	4
7.3	Ordered collections	4
8	Compatibility	4

1 Scope

DIPH5 is the HDF5 interchange format used by SciNumTools to persist an *evaluated DIPL environment*. It stores evaluated value nodes together with their DIPL paths, collection structure, units, settings, and provenance.

The persisted object is therefore a data environment suitable for exchange with scientific applications. It is not a serialization of the complete parser or evaluation runtime.

This document specifies the on-disk representation. It does not define the DIPL language itself; the DIPL specification defines the language concepts represented here.

2 Persisted environment boundary

The file contains the evaluated value-node tree and the node-level information needed to interpret and trace those values. In particular, node settings, units, schemas, source information, and provenance are persisted as dataset attributes.

Implementation-specific runtime registries are not serialized as independent objects. This includes global function definitions, branching state, and the complete source, unit, and schema registries. Information from those registries that belongs to an individual evaluated node is retained on that node where possible. A loaded DIPH5 file must therefore be treated as an evaluated environment, not as a source from which the original DIPL program can be reconstructed exactly.

3 File identification and versioning

The HDF5 root object MUST contain the following scalar attributes:

Attribute	Type	Required value
<code>_DIPL_Format</code>	UTF-8 string	SciNumTools3 Environment
<code>_DIPL_Schema_Version</code>	unsigned integer	1

Readers MUST reject files with a different format identifier or unsupported schema version. Future schema revisions MUST preserve the meaning of existing attributes or increment the schema version.

3.1 File naming

Files SHOULD use the `.diph5` filename extension to make their purpose clear. The extension is only a naming convention and is not normative. Readers MUST identify a file from the root `_DIPL_Format` and `_DIPL_Schema_Version` attributes, regardless of its filename.

4 Paths and objects

Each component of a fully qualified DIPL path is represented by an HDF5 path component. A value node is an HDF5 dataset at its resolved path. Intermediate groups represent DIPL hierarchy and carry `_DIPL_Kind=group` and `_DIPL_Path` attributes.

The `_DIPL_Path` attribute stores the corresponding fully qualified DIPL path and is the authoritative mapping back to DIPL semantics.

4.1 Collections

Collection containers and items are represented by groups. Their meaning is identified by `_DIPL_Kind`:

<code>_DIPL_Kind</code>	Meaning	Additional attributes
<code>map</code>	keyed collection	—
<code>map_item</code>	item of a keyed collection	<code>_DIPL_Key</code>
<code>list</code>	ordered collection	—
<code>list_item</code>	item of an ordered collection	<code>_DIPL_Index</code>

Collection item groups may contain nested groups and value datasets.

The complete object-kind table is:

HDF5 object	<code>_DIPL_Kind</code>	<code>_DIPL_Path</code>	Other required attributes
hierarchy group	<code>group</code>	group path	—
keyed collection	<code>map</code>	collection path	—
keyed item	<code>map_item</code>	item path	<code>_DIPL_Key</code>
ordered collection	<code>list</code>	collection path	—
ordered item	<code>list_item</code>	item path	<code>_DIPL_Index</code>
value dataset	<code>value</code>	value path	<code>_DIPL_Node_Type</code> , <code>_DIPL_Value_Type</code>

An implementation MUST use the kind attribute to distinguish a collection from an ordinary hierarchy group. HDF5 object names alone are not sufficient, because a numbered or keyed collection item is deliberately flattened into a normal group path.

5 Value datasets

Every value dataset has `_DIPL_Kind=value`, `_DIPL_Path`, `_DIPL_Node_Type`, and `_DIPL_Value_Type` attributes. `_DIPL_Array` identifies array values; scalar values use a scalar dataspace and arrays use a simple dataspace whose dimensions match the DIPL value shape.

Values are written using native HDF5 numeric types. Strings use fixed-width UTF-8 HDF5 string datatypes. A null DIPL value is represented by a dataset with an HDF5 null dataspace.

5.1 Value type table

The `_DIPL_Value_Type` value identifies the logical DIPL type. The following table lists the concrete types currently written by DIP5 and their HDF5 storage types:

<code>_DIPL_Value_Type</code>	Logical value	HDF5 datatype	Scalar/array
<code>bool</code>	Boolean	native unsigned 8-bit integer	both
<code>char</code>	Character	native signed 8-bit integer	both
<code>int8</code> , <code>int16</code> , <code>int32</code> , <code>int64</code>	Signed integer	matching native signed integer	both
<code>uint8</code> , <code>uint16</code> , <code>uint32</code> , <code>uint64</code>	Unsigned integer	matching native unsigned integer	both
<code>float32</code>	Single precision real	native 32-bit float	both
<code>float64</code>	Double precision real	native 64-bit float	both
<code>float128</code>	Extended precision real	native long double	both
<code>str</code>	UTF-8 string	fixed-width UTF-8 string	both

For a scalar, the dataset has an HDF5 scalar dataspace. For an array, the dataset has a simple dataspace and `_DIPL_Array` is non-zero. The dimensions are stored in the dataspace rather than in a separate attribute.

6 Node settings and provenance

When present, node settings are stored as attributes on the value dataset:

`_DIPL_Constant`, `_DIPL_Condition`, `_DIPL_Format_Spec`, `_DIPL_Tags`, `_DIPL_Options`, `_DIPL_Option_Units`, `_DIPL_Schemas`, and `_DIPL_Value-Origin`.

Source and provenance information uses `_DIPL_Source`, `_DIPL_Source_Line`, `_DIPL_Source_Code`, and the metadata attributes `description`, `authors`, `title`, `journal`, `year`, `volume`, `issue`, `pages`, `doi`, `url`, `version`, `created`, `modified`, and `license`. Physical units are stored in the `units` attribute using their canonical PUEL representation.

7 Examples

7.1 Scalar values and hierarchy

This DIPL fragment:

```
simulation
  steps int = 100
  enabled bool = true
```

is represented as:

```
/simulation          group  (_DIPL_Kind=group)
/simulation/steps     dataset (_DIPL_Kind=value,
                        _DIPL_Path=simulation.steps,
                        _DIPL_Value_Type=int32)
/simulation/enabled   dataset (_DIPL_Kind=value,
                        _DIPL_Path=simulation.enabled,
                        _DIPL_Value_Type=bool)
```

/simulation/steps has a scalar dataspace and a native signed 32-bit HDF5 datatype. /simulation/enabled has a scalar dataspace and a native unsigned 8-bit HDF5 datatype.

7.2 Arrays and units

This DIPL value:

```
boundary[inlet]
  velocity float[3] = [1.0, 0.0, 0.0] m/s
```

is represented as:

/boundary	group	(_DIPL_Kind=map, _DIPL_Path=boundary)
/boundary/inlet	group	(_DIPL_Kind=map_item, _DIPL_Path=boundary[inlet], _DIPL_Key=inlet)
/boundary/inlet/velocity	dataset	(_DIPL_Kind=value, _DIPL_Path=boundary[inlet].velocity, _DIPL_Value_Type=float64, _DIPL_Array=1, units=m*s-1)

The velocity dataset has rank one and dimension 3.

7.3 Ordered collections

This DIPL fragment:

```
samples[]
  time float = 0 s
samples[]
  time float = 1 s
```

is represented by a list group containing list_item groups:

/samples	group	(_DIPL_Kind=list, _DIPL_Path=samples)
/samples/0	group	(_DIPL_Kind=list_item, _DIPL_Path=samples[0], _DIPL_Index=0)
/samples/0/time	dataset	(_DIPL_Path=samples[0].time)
/samples/1	group	(_DIPL_Kind=list_item, _DIPL_Path=samples[1], _DIPL_Index=1)
/samples/1/time	dataset	(_DIPL_Path=samples[1].time)

The _DIPL_Index attribute, rather than HDF5 group iteration order, defines the order of list items.

8 Compatibility

Implementations SHOULD preserve unknown attributes when editing a DIP5 file and MUST ignore attributes they do not understand. They MUST reject a file whose format identifier or schema version is unsupported rather than silently interpreting it as a different format.